

A. 排列 (perm)

核心思路

设 $\text{pos}_q(v)$ 表示数值 v 在排列 q 中的位置，构造排列

$$r_i = \text{pos}_q(p_i). \quad (1)$$

原问题等价于：通过恰好 k 次交换，将排列 r 变为单位排列。

设 r 的置换环数量为 c ，其中长度为 1 的环也计入，则最少交换次数为

$$d = n - c. \quad (2)$$

由于题目保证 $n \geq 2$ ，答案为 `yes` 当且仅当

$$k \geq d \quad \text{且} \quad (k - d) \bmod 2 = 0. \quad (3)$$

正确性说明

一次交换会将一个置换环拆成两个，或者将两个置换环合并成一个，因此环数每次恰好变化 1。

单位排列有 n 个环，所以至少需要 $n - c$ 次交换。对于一个长度为 s 的环，可以用 $s - 1$ 次交换将其拆成单点环，故该下界可以达到。

同时，每次交换都会改变环数的奇偶性，所以所有可行操作次数都与 $n - c$ 同奇偶。

在最少交换方案之后，可以反复对同一对不同位置连续交换两次，每次增加 2 次操作而不改变排列。因此上述条件也是充分的。

B. 混乱善良 (cross)

将两个数组中的元素看作数轴上的两类点，匹配 a_i 与 b_{p_i} 。

由于两个数组均严格递增，按相同下标匹配可以使距离和最小。因此最小距离和为

$$\sum_{i=1}^n |a_i - b_i|. \quad (4)$$

现在我们考虑一个特殊情况：所有 $a_i \leq b_i$ 。这时候距离和最小的充要条件是：所有 a_i 都向后匹配，所有 b_i 都向前匹配。按坐标从小到大归并两个数组，若两个坐标相同则属于 a 数组的排在前面。若将归并后的序列中，属于 a 的视为左括号，属于 b 的视为右括号。则逆序对数最多的匹配一定就是这个括号序列的**括号匹配**（由于 $a_i < b_i$ 所以括号序列一定合法）。具体可以通过调整法来证明。

也就是我们可以用栈去维护这个过程，甚至我们不需要栈，只需要维护当前没有匹配的左括号数量，即可计算答案。

推广到一般情况，把 $1 \sim n$ 分成若干极长 $a_i \leq b_i$ 或 $a_i \geq b_i$ 的连续段。每个连续段之间互不影响，对所有连续段单独处理即可。

实际上也不用划分连续段，维护变量 h 表示栈的状态：

- $h < 0$: 栈中剩下 $|h|$ 个 b_i 没有匹配;
- $h > 0$: 栈中剩下 h 个 a_i 没有匹配。
- $h = 0$: 栈为空。

维护归并的过程, 设当前扫到了 a_i 和 b_j :

- $a_i < b_j$, 加入 a_i , $ans \leftarrow ans + \max(0, -(h + 1))$, $h \leftarrow h + 1$ 。
- $a_i > b_j$, 加入 b_j , $ans \leftarrow ans + \max(0, h - 1)$, $h \leftarrow h - 1$ 。
- $a_i = b_j$, 同时加入 a_i, b_j , 并强制要求它们匹配, $ans \leftarrow ans + |h|$, h 不变。

记得开 longlong。

C. 挑战 LIS (lis)

原题: <https://qoj.ac/problem/16005>。

考虑用数据结构维护两个数组 f, g , 其中 f_i 表示管理员左边, 结尾为 i 的最长上升子序列长度, g_i 表示管理员右边, 开头为 j 的最长上升子序列长度。

则整个序列的最长上升子序列长度为:

$$\max_{i < j} f_i + g_j \quad (5)$$

对于两个数组, 我们只需要维护:

- 单点修改
- 区间求 max
- 回滚到上一个版本

使用线段树维护, 对于回滚, 记录操作序列或在叶子结点维护 multiset 均可。

将这两个数组放到一颗线段树上, 则答案也是可以通过左右区间合并直接维护的信息。具体地, 设 ans_p 表示 p 节点的区间下, $\max_{l \leq i < j \leq r} f_i + g_j$ 的值, 同时维护 $maxf_p, maxg_p$ 。则区间合并时:

$$ans_p = \max(ans_{ls_p}, ans_{rs_p}, maxf_{ls_p} + maxg_{rs_p}) \quad (6)$$

时间复杂度 $O((n + q) \log V)$ 。

D. 前缀和 (sum)

题目来源: ERoRain。

下文假设 n, V 同阶。

算法一

枚举 P, b, c , 表示我们想要将前缀中恰好 c 个元素替换为前缀外的元素, 使得 $s_P = b \in B$ 。

设 $f_{i,j,k}$ 表示前缀 $[1, i]$ 中, 挖掉恰好 j 个元素能否使总和为 k , $g_{i,j,k}$ 表示后缀 $[i, n]$ 中, 选择恰好 j 个元素能否使总和为 k 。

若存在 v ，使得 $f_{P,c,v} = g_{P+1,c,b-v} = 1$ ，则存在一种交换次数为 c 的方案。

处理 f, g 复杂度为 $O(n^3)$ ，计算答案复杂度 $O(n^4)$ 。

算法二

使用 bitset 优化，复杂度 $O(\frac{n^4}{w})$ 。

算法三

考虑不枚举 b ，我们实际想做的是判断 $f_{i,c} * g_{i+1,c} \cap B$ 是否非空（* 是多项式卷积），考虑把 B 放在 f 的初始化中，更一般化地解释这个东西：

我们想要计算下列式子

$$\sum ([x^i]F * G) \times h_i \quad (7)$$

设多项式 $H = \sum h_i x^{-i}$ ，则原式等于：

$$[x^0]F * G * H = [x^0](F * H) * G \quad (8)$$

如果能提前计算 $F * H$ ，我们就可以把求两个多项式卷积的所有项，变成了求两个多项式卷积后的一项。

回到这道题，由于可达性背包也可以视作多项式， h_i 就是 $[i \in B]$ ， $H = \sum [i \in B]x^{-i}$ 。我们想要把 H 提前卷到 F 上，也就是在初始化时，让 $f_{0,0,-i} = [i \in B]$ 。由于负数不太好做，初始化改成 $f_{0,0,i} = [i \in B]$ ，做背包的时候倒着转移（即 $f_{i,j,k} \rightarrow f_{i+1,j,k-a_{i+1}}$ ），最后只需要判断是否有 $f_{P,c,v} = g_{P+1,c,v} = 1$ 即可，这一部分也可以使用 bitset。

复杂度 $O(\frac{n^3}{w})$ 。

算法四

暂时忘掉算法三。

考虑不枚举 c ，设某种方案中选择的下标集合为 U ， $h_i = [i \in U]$ ，则最小交换次数为：

$$\min_{p'} \{ \max(p' - \sum_{i=1}^{p'} h_i, \sum_{i=p'+1}^n h_i) \} \quad (9)$$

于是我们考虑把 c 记到 dp 值上， $f_{i,k}$ 表示前缀 $[1, i]$ 中使总和为 k 最少需要挖掉多少元素， $g_{i,k}$ 表示后缀 $[i, n]$ 中，使总和为 k 最少需要选多少个元素。

则枚举 P, b ，将 $\min_v \{ \max(f_{P,v}, g_{P+1,b-v}) \}$ 贡献到答案。

可以证明这样做不会漏掉最优解。

复杂度 $O(n^3)$ 。

算法五

将算法四和算法三结合起来，即最优化背包也可以用类似的方法，把 B 集合提前初始化到 f 上。具体地：

$$f_{0,i} = \begin{cases} 0, & i \in B, \\ +\infty, & i \notin B. \end{cases} \quad (10)$$

此时 f 的状态更改为：处理前 i 个元素后，保留元素的总和距离某个目标值还差 x 时，最少需要删除多少个元素。

再倒着做最优化背包即可，转移为 $f_{i,x} = \min(f_{i-1,x} + 1, f_{i-1,x+A_i})$ 。注意判无解。

复杂度 $O(n^2)$ 。